M1 Proposal — Mobile-Wallet Fraud Detection

Team number: ___
Supervisors: Dr. Ghada Khoriba (CO 208), Dr. Amal Foad (AI 306), Dr. Abdulla Mamdouh (CS 312)
Course: Integrated Mega Project, Fall 2026, School of Computing and Data Science, Badya University

#	Member	ID	Lead role
1	Abdelrahman Ahmed	2400736	Database and web lead
2	Fathy Ahmed	2400740	Data and statistics lead
3	Nasr Yousef	2400825	AI lead
4	Adham Mohamed	2400517	Project manager + Quality and documentation lead

1. Problem statement

Fraud in mobile wallets and instant transfers is growing, and its victims are ordinary users who lose their balance within minutes. Providers cannot review millions of transactions by hand, so they need a system that flags suspicious ones for review first. The difficulty is that fraud is under 1% of transactions, so a model that never flags anything looks 99%+ accurate while catching nothing. We build a fraud-screening web application that is judged on precision, recall and F1, not accuracy, and that explains why each transaction was flagged.

2. Objectives

1. Build an automated, tested Python pipeline that loads, cleans and validates a stratified sample of at least 100,000 transactions into a relational database.
2. Answer at least 2 statistical questions with confidence intervals and hypothesis tests, with assumptions checked.
3. Reach an F1 score on the fraud class that is clearly above the "never fraud" baseline, with recall of at least 80% on a held-out test set.
4. Deliver a web application where a non-technical user enters a transaction and gets a verdict with the reasons behind it.

3. Data source

PaySim, a public synthetic mobile-money dataset on Kaggle (`ealaxi/paysim1`), generated from aggregated logs of a real African mobile-money service (Lopez-Rojas, Elmahdy and Axelsson, 2016). It has about 6.36 million records and 11 variables: step (hour), type, amount, sender and receiver IDs, sender and receiver balances before and after, and the fraud label. No real personal data is involved. Licence: CC BY-SA 4.0 as listed on Kaggle.

We will use all fraud rows plus a random sample of legitimate rows (about 100,000 or more), keeping a separate test set at the natural fraud rate. The data is simulated and not Egyptian, and we will say so in the report and justify it as the closest open dataset for this problem.

4. Statistical questions

1. Are fraudulent transactions larger than legitimate ones? A 95% confidence interval for the difference in mean log(amount), a Welch t-test on log(amount) after checking normality, and a Mann-Whitney U test as the non-parametric check.
2. Is transaction type associated with fraud? A chi-square test of independence between type and fraud, plus a Wilson 95% confidence interval for the fraud rate in each type.
3. Which features predict fraud? A logistic regression on amount, balance-to-amount ratio and type, with a fitted log-normal distribution for amount (Q-Q plot and Kolmogorov-Smirnov check).

5. AI component

Technique (from the course): Bayesian reasoning combined with logic-based rules. A Naive Bayes classifier, implemented from scratch, gives the probability of fraud from discretized features (type, amount band, share of balance moved, receiver balance was zero). Explicit rules such as "TRANSFER or CASH_OUT that empties the sender's balance" add or override evidence and are shown to the user as the reason for a flag.

Input and output: a transaction's type, amount, sender and receiver balances; the output is a verdict (safe or suspicious), a fraud probability, and the rules and features that drove it.

Baselines: (1) always predict "not fraud", which scores over 99% accuracy but zero recall; (2) rules only, without Bayes. We evaluate with a stratified train/test split, precision, recall, F1, a precision-recall curve and an analysis of false positives and false negatives.

6. System sketch

Web pages (Flask or FastAPI):

1. Dashboard: charts of fraud by type, amount distribution and activity over time.
2. Try the AI: enter a transaction and get a verdict with reasons.
3. Statistics: confidence intervals, test results and their interpretation.
4. About the model: metrics against the baseline, errors and limitations.

Main database entities (SQLite or PostgreSQL, third normal form): transactions, accounts, transaction_types, fraud_labels, predictions_log.

7. Risks and plan B

Risk	Plan B
The full 6.4 million rows are too slow to load or analyze	Work on the stratified sample and load in chunks
Fraud is very rare, so Naive Bayes has low recall	Add engineered features (balance-to-amount ratio, zero receiver balance), reweight classes, and give the rules more weight

Risk	Plan B
The data is synthetic and not Egyptian	State it openly and justify it; add a note on how real data would change results
A false positive could block an honest user	Report the false-positive rate and present the output as a flag for human review, never an automatic block
A team member falls behind	Weekly sprint review; every member commits through their own pull requests